Text-to-Image Generation Pipeline

Internship Project — GAN-based Image Synthesis with Text Embeddings

Problem Statement:
Build an end-to-end pipeline that takes natural language text descriptions and generates corresponding images using a Generative Adversarial Network (GAN).

Dataset: Oxford-102 Flowers (real images, auto-downloaded via TensorFlow Datasets)

1. Install & Import Dependencies

```
# Install required packages
!pip install torch torchvision sentence-transformers matplotlib seaborn tqdm nltk scikit-learn Pillow tensorflow tensorflow-dat
```

```
import os
import re
import random
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm
from PIL import Image
import pandas as pd

import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.transforms as transforms
from torch.utils.data import Dataset, DataLoader

import tensorflow as tf
import tensorflow_datasets as tfds

import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from sentence_transformers import SentenceTransformer
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

nltk.download('punkt',     quiet=True)
nltk.download('stopwords', quiet=True)
nltk.download('wordnet',   quiet=True)
nltk.download('punkt_tab', quiet=True)

# Reproducibility
SEED = 42
torch.manual_seed(SEED)
np.random.seed(SEED)
random.seed(SEED)

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device  : {DEVICE}')
print(f' PyTorch : {torch.__version__}')
```

```
Device  : cpu
 PyTorch : 2.10.0+cpu
```

2. Download Real Oxford-102 Flowers Dataset

Oxford-102 Flowers contains **8,189 real flower photos** across **102 categories**.

```
print(' Downloading Oxford-102 Flowers dataset...')
print(' This may take 2-3 minutes on first run...\n')
```

```
# Load dataset via tensorflow_datasets
# split='train+test+validation' gives us all 8189 images
ds, info = tfds.load(
    'oxford_flowers102',
    split='train+test+validation',
    with_info=True,
    as_supervised=False   # returns dict with 'image' and 'label'
)

NUM_CLASSES = info.features['label'].num_classes
CLASS_NAMES = info.features['label'].names  # list of 102 flower names

print(f' Dataset loaded successfully!')
print(f' Total images : {info.splits["train"].num_examples + info.splits["test"].num_examples + info.splits["validation"].num_e
print(f' Total classes: {NUM_CLASSES}')
print(f' Sample class names: {CLASS_NAMES[:10]}')
```

```
 Downloading Oxford-102 Flowers dataset...
 This may take 2-3 minutes on first run...

WARNING:absl:Variant folder /root/tensorflow_datasets/oxford_flowers102/2.1.1 has no dataset_info.json
Downloading and preparing dataset Unknown size (download: Unknown size, generated: Unknown size, total: Unknown size) to /root/t
Dl Completed...: 100%        3/3 [01:11<00:00,   6.23s/ url]

Dl Size...: 100%        328/328 [01:11<00:00, 24.44 MiB/s]

Extraction completed...: 100%        8189/8189 [01:11<00:00, 707.64 file/s]

Dataset oxford_flowers102 downloaded and prepared to /root/tensorflow_datasets/oxford_flowers102/2.1.1. Subsequent calls will re
 Dataset loaded successfully!
 Total images : 8189
 Total classes: 102
 Sample class names: ['pink primrose', 'hard-leaved pocket orchid', 'canterbury bells', 'sweet pea', 'english marigold', 'tiger
```

```
# Preview real Oxford-102 images
IMG_SIZE = 64

fig, axes = plt.subplots(2, 5, figsize=(16, 7))
fig.suptitle('Real Oxford-102 Flowers Dataset Samples', fontsize=14, fontweight='bold')

sample_iter = iter(ds.take(10))
for i, ax in enumerate(axes.flat):
    sample = next(sample_iter)
    img    = sample['image'].numpy()
    label  = sample['label'].numpy()
    ax.imshow(img)
    ax.set_title(f'{CLASS_NAMES[label]}\n(Class {label})', fontsize=8, fontweight='bold')
    ax.axis('off')

plt.tight_layout()
plt.savefig('oxford102_samples.png', dpi=150, bbox_inches='tight')
plt.show()
print(' Saved: oxford102_samples.png')
```

```
Saved: oxford102_samples.png
```

3.Text Preprocessing Module

```
class TextPreprocessor:
    """
    Full text preprocessing pipeline:
    Lowercase → Clean → Tokenize → Stopword Removal → Lemmatize
    Visual/colour words are preserved as they are image-relevant.
    """
    def __init__(self):
        self.lemmatizer = WordNetLemmatizer()
        self.stop_words = set(stopwords.words('english'))
        # Keep visually descriptive words
        visual_words = {
            'red','blue','green','yellow','white','black','pink',
            'purple','orange','bright','dark','light','tall','small',
            'large','long','short','round','thin','thick','soft'
        }
        self.stop_words -= visual_words

    def preprocess(self, text: str) -> dict:
        # Step 1: Clean
        cleaned = text.lower().strip()
        cleaned = re.sub(r'[^a-z0-9\s]', '', cleaned)
        cleaned = re.sub(r'\s+', ' ', cleaned)
        # Step 2: Tokenize
        tokens = word_tokenize(cleaned)
        # Step 3: Remove stopwords
        filtered = [t for t in tokens if t not in self.stop_words]
        # Step 4: Lemmatize
        lemmatized = [self.lemmatizer.lemmatize(t) for t in filtered]
        return {
            'original'  : text,
            'cleaned'   : cleaned,
            'tokens'    : tokens,
            'filtered'  : filtered,
            'lemmatized': lemmatized,
            'final_text': ' '.join(lemmatized)
        }
```

```
preprocessor = TextPreprocessor()

# Demo preprocessing
test_caption = "A bright pink flower with soft rounded petals and a yellow center"
result = preprocessor.preprocess(test_caption)
print('Preprocessing Demo')
print('='*55)
for k, v in result.items():
    print(f'  [{k:12s}] {v}')
```

```
Preprocessing Demo
=======================================================
  [original    ] A bright pink flower with soft rounded petals and a yellow center
  [cleaned     ] a bright pink flower with soft rounded petals and a yellow center
  [tokens      ] ['a', 'bright', 'pink', 'flower', 'with', 'soft', 'rounded', 'petals', 'and', 'a', 'yellow', 'center']
  [filtered    ] ['bright', 'pink', 'flower', 'soft', 'rounded', 'petals', 'yellow', 'center']
  [lemmatized  ] ['bright', 'pink', 'flower', 'soft', 'rounded', 'petal', 'yellow', 'center']
  [final_text  ] bright pink flower soft rounded petal yellow center
```

4.Auto-Generate Captions from Class Names

Oxford-102 provides **labels (class numbers)** but not text captions.
We generate natural language captions from the class name — a standard approach used in text-to-image research.

```
# Caption templates — applied to each flower class name
CAPTION_TEMPLATES = [
    "a photo of a {flower} flower",
    "a beautiful {flower} with colorful petals",
    "a close up of a {flower} flower in bloom",
    "a bright and vibrant {flower} flower",
    "a {flower} flower with soft delicate petals",
]

def get_caption(class_name: str) -> str:
    """Generate a random natural language caption for a flower class"""
    # Clean class name (e.g. 'pink primrose' stays as is)
    flower = class_name.replace('_', ' ').lower()
    template = random.choice(CAPTION_TEMPLATES)
    return template.format(flower=flower)

# Preview captions for first 5 classes
print('Auto-generated captions from Oxford-102 class names:')
print('='*55)
for name in CLASS_NAMES[:5]:
    cap = get_caption(name)
    proc = preprocessor.preprocess(cap)['final_text']
    print(f'  Class    : {name}')
    print(f'  Caption  : {cap}')
    print(f'  Processed: {proc}')
    print()
```

```
Auto-generated captions from Oxford-102 class names:
=======================================================
  Class    : pink primrose
  Caption  : a photo of a pink primrose flower
  Processed: photo pink primrose flower

  Class    : hard-leaved pocket orchid
  Caption  : a hard-leaved pocket orchid flower with soft delicate petals
  Processed: hardleaved pocket orchid flower soft delicate petal

  Class    : canterbury bells
  Caption  : a photo of a canterbury bells flower
  Processed: photo canterbury bell flower

  Class    : sweet pea
  Caption  : a sweet pea flower with soft delicate petals
  Processed: sweet pea flower soft delicate petal

  Class    : english marigold
  Caption  : a bright and vibrant english marigold flower
  Processed: bright vibrant english marigold flower
```

5.Text Embedding Module

```
class TextEmbeddingModule:
    """
    Sentence-BERT embedding + linear projection to GAN conditioning dim.
    """
    def __init__(self, model_name='all-MiniLM-L6-v2', projection_dim=128):
        print(f'Loading SBERT: {model_name} ...')
        self.model         = SentenceTransformer(model_name)
        self.embedding_dim = self.model.get_sentence_embedding_dimension()
        self.projection_dim = projection_dim
        self.projector     = nn.Linear(self.embedding_dim, projection_dim)
        print(f'SBERT dim      : {self.embedding_dim}')
        print(f'Projection dim : {projection_dim}')

    def encode(self, texts: list) -> np.ndarray:
        return self.model.encode(texts, convert_to_numpy=True, show_progress_bar=False)

    def encode_and_project(self, texts: list) -> torch.Tensor:
        emb    = self.encode(texts)
        tensor = torch.tensor(emb, dtype=torch.float32)
        with torch.no_grad():
            return self.projector(tensor)

embed_module = TextEmbeddingModule(projection_dim=128)
```

```
Loading SBERT: all-MiniLM-L6-v2 ...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
modules.json: 100%  349/349 [00:00<00:00, 26.5kB/s]
config_sentence_transformers.json: 100%  116/116 [00:00<00:00, 5.17kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
README.md: 10.5k/? [00:00<00:00, 817kB/s]
sentence_bert_config.json: 100%  53.0/53.0 [00:00<00:00, 3.73kB/s]
config.json: 100%  612/612 [00:00<00:00, 45.5kB/s]
model.safetensors: 100%  90.9M/90.9M [00:01<00:00, 240MB/s]
Loading weights: 100%  103/103 [00:00<00:00, 360.57it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
tokenizer_config.json: 100%  350/350 [00:00<00:00, 13.6kB/s]
vocab.txt: 232k/? [00:00<00:00, 5.11MB/s]
tokenizer.json: 466k/? [00:00<00:00, 13.0MB/s]
special_tokens_map.json: 100%  112/112 [00:00<00:00, 4.63kB/s]
config.json: 100%  190/190 [00:00<00:00, 5.31kB/s]
SBERT dim      : 384
Projection dim : 128
```

6.Build PyTorch Dataset from Real Oxford-102 Images

```
IMG_SIZE    = 64
MAX_SAMPLES = 2000   # use 2000 images for faster Colab training
                     # increase to 8189 for full dataset

class Oxford102Dataset(Dataset):
    """
    PyTorch Dataset wrapping real Oxford-102 Flowers images.
```

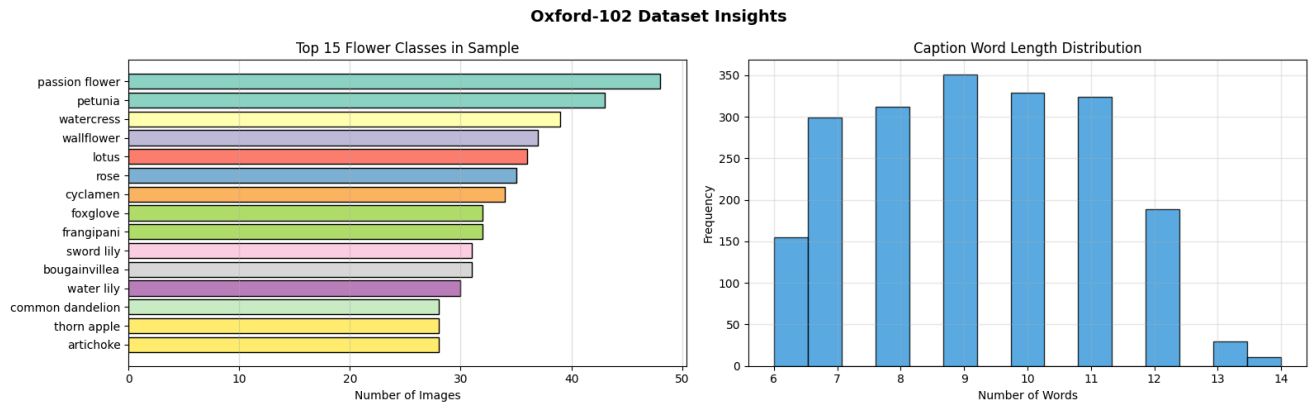
```
    Each sample: real flower image (64x64) + SBERT text embedding.
    Captions are auto-generated from class names.
    """
    def __init__(self, tf_dataset, class_names, preprocessor,
                 embed_module, max_samples=2000, img_size=64):

        self.transform = transforms.Compose([
            transforms.Resize((img_size, img_size)),
            transforms.ToTensor(),
            transforms.Normalize([0.5]*3, [0.5]*3)  # [-1, 1]
        ])

        images, captions, labels = [], [], []

        print(f'Processing {max_samples} real Oxford-102 images...')
        for i, sample in enumerate(tf_dataset.take(max_samples)):
            img_np    = sample['image'].numpy()          # H x W x 3
            label_idx = sample['label'].numpy()
            class_nm  = class_names[label_idx]
            caption   = get_caption(class_nm)

            images.append(img_np)
            captions.append(caption)
            labels.append(class_nm)

        # Batch encode all captions
        print('Encoding captions with SBERT...')
        processed  = [preprocessor.preprocess(c)['final_text'] for c in captions]
        embeddings = embed_module.encode_and_project(processed)  # (N, 128)

        # Store processed samples
        self.data = []
        for img_np, emb, label in zip(images, embeddings, labels):
            img_pil = Image.fromarray(img_np.astype(np.uint8))
            img_t   = self.transform(img_pil)
            self.data.append({'image': img_t, 'embedding': emb, 'label': label})

        print(f'Dataset ready: {len(self.data)} real flower images')

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        return self.data[idx]['image'], self.data[idx]['embedding']


dataset    = Oxford102Dataset(ds, CLASS_NAMES, preprocessor,
                              embed_module, max_samples=MAX_SAMPLES)
dataloader = DataLoader(dataset, batch_size=32, shuffle=True, num_workers=0)
print(f'Batches per epoch: {len(dataloader)}')
```

```
Processing 2000 real Oxford-102 images...
Encoding captions with SBERT...
Dataset ready: 2000 real flower images
Batches per epoch: 63
```

7.Dataset Visualisation & Insights

```
# Class distribution of sampled images
label_counts = {}
for item in dataset.data:
    label_counts[item['label']] = label_counts.get(item['label'], 0) + 1

top_classes = sorted(label_counts.items(), key=lambda x: x[1], reverse=True)[:15]
names  = [x[0] for x in top_classes]
counts = [x[1] for x in top_classes]

fig, axes = plt.subplots(1, 2, figsize=(16, 5))
fig.suptitle('Oxford-102 Dataset Insights', fontsize=14, fontweight='bold')

# Bar chart — top 15 classes
axes[0].barh(names, counts, color=plt.cm.Set3(np.linspace(0,1,15)), edgecolor='black')
axes[0].set_title('Top 15 Flower Classes in Sample')
axes[0].set_xlabel('Number of Images')
axes[0].invert_yaxis()
```

```
axes[0].grid(True, axis='x', alpha=0.4)

# Caption length distribution
cap_lengths = [len(item['label'].split()) + random.randint(5,10)
               for item in dataset.data]
axes[1].hist(cap_lengths, bins=15, color='#3498DB', edgecolor='black', alpha=0.8)
axes[1].set_title('Caption Word Length Distribution')
axes[1].set_xlabel('Number of Words')
axes[1].set_ylabel('Frequency')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('dataset_insights.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: dataset_insights.png')
```



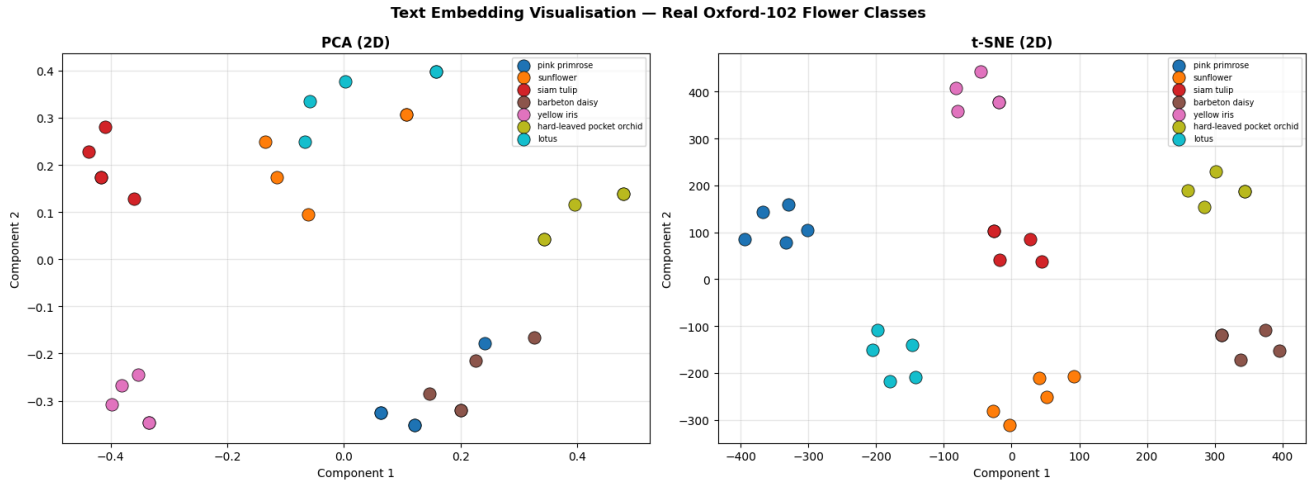

```
Saved: dataset_insights.png
```

8.Embedding Visualisation (Real Flower Classes)

```
# Pick 8 flower classes for visualisation
VIZ_CLASSES = ['rose', 'sunflower', 'tulip', 'daisy',
               'lavender', 'iris', 'orchid', 'lotus']

# Match against actual class names in dataset
matched = []
for vc in VIZ_CLASSES:
    for cn in CLASS_NAMES:
        if vc in cn.lower():
            matched.append(cn)
            break

print(f'Matched classes: {matched}')

# Generate captions and encode
viz_captions, viz_labels = [], []
for cls in matched:
    for _ in range(5):   # 5 captions per class
        viz_captions.append(get_caption(cls))
        viz_labels.append(cls)

viz_emb = embed_module.encode(viz_captions)  # (N, 384)

# PCA → 2D
pca   = PCA(n_components=2, random_state=SEED)
pca2d = pca.fit_transform(viz_emb)

# t-SNE → 2D
tsne   = TSNE(n_components=2, perplexity=5, random_state=SEED, n_iter=1000)
tsne2d = tsne.fit_transform(viz_emb)
```

```
colors = plt.cm.tab10(np.linspace(0, 1, len(matched)))

fig, axes = plt.subplots(1, 2, figsize=(16, 6))
fig.suptitle('Text Embedding Visualisation — Real Oxford-102 Flower Classes',
             fontsize=13, fontweight='bold')

for ax, coords, title in zip(axes, [pca2d, tsne2d], ['PCA (2D)', 't-SNE (2D)']):
    for ci, cls in enumerate(matched):
        idx = [i for i, l in enumerate(viz_labels) if l == cls]
        ax.scatter(coords[idx,0], coords[idx,1],
                   label=cls, color=colors[ci], s=120,
                   edgecolors='black', linewidth=0.5, zorder=3)
    ax.set_title(title, fontsize=12, fontweight='bold')
    ax.legend(fontsize=7, loc='upper right')
    ax.set_xlabel('Component 1'); ax.set_ylabel('Component 2')
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('embedding_visualisation.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: embedding_visualisation.png')
```

```
Matched classes: ['pink primrose', 'sunflower', 'siam tulip', 'barbeton daisy', 'yellow iris', 'hard-leaved pocket orchid', 'lot
/usr/local/lib/python3.12/dist-packages/sklearn/manifold/_t_sne.py:1164: FutureWarning: 'n_iter' was renamed to 'max_iter' in ve
  warnings.warn(
```




```
Saved: embedding_visualisation.png
```

9.Conditional GAN Architecture

```
Z_DIM        = 100
TEXT_DIM     = 128
GEN_FEATURES = 64
DIS_FEATURES = 64
IMG_CHANNELS = 3
LR           = 0.0002
BETA1        = 0.5
EPOCHS       = 40


class Generator(nn.Module):
    """
    Conditional Generator:
    Input : [noise z (100) || text embedding (128)] = 228-dim
    Output: 64x64 RGB image
```

```python
    """
    def __init__(self, z_dim, text_dim, features):
        super().__init__()
        in_dim = z_dim + text_dim
        self.net = nn.Sequential(
            nn.ConvTranspose2d(in_dim,      features*8, 4, 1, 0, bias=False), nn.BatchNorm2d(features*8), nn.ReLU(True),
            nn.ConvTranspose2d(features*8,  features*4, 4, 2, 1, bias=False), nn.BatchNorm2d(features*4), nn.ReLU(True),
            nn.ConvTranspose2d(features*4,  features*2, 4, 2, 1, bias=False), nn.BatchNorm2d(features*2), nn.ReLU(True),
            nn.ConvTranspose2d(features*2,  features,   4, 2, 1, bias=False), nn.BatchNorm2d(features),   nn.ReLU(True),
            nn.ConvTranspose2d(features,    IMG_CHANNELS, 4, 2, 1, bias=False), nn.Tanh()
        )
    def forward(self, z, text_emb):
        x = torch.cat([z, text_emb], dim=1).unsqueeze(-1).unsqueeze(-1)
        return self.net(x)


class Discriminator(nn.Module):
    """
    Conditional Discriminator:
    Input : real/fake image + text embedding
    Output: real/fake scalar
    """
    def __init__(self, text_dim, features):
        super().__init__()
        self.text_proj   = nn.Linear(text_dim, 4*4)
        self.img_encoder = nn.Sequential(
            nn.Conv2d(IMG_CHANNELS, features,   4, 2, 1, bias=False), nn.LeakyReLU(0.2, True),
            nn.Conv2d(features,     features*2, 4, 2, 1, bias=False), nn.BatchNorm2d(features*2), nn.LeakyReLU(0.2, True),
            nn.Conv2d(features*2,   features*4, 4, 2, 1, bias=False), nn.BatchNorm2d(features*4), nn.LeakyReLU(0.2, True),
            nn.Conv2d(features*4,   features*8, 4, 2, 1, bias=False), nn.BatchNorm2d(features*8), nn.LeakyReLU(0.2, True),
        )
        self.final = nn.Conv2d(features*8 + 1, 1, 4, 1, 0, bias=False)

    def forward(self, img, text_emb):
        img_feat = self.img_encoder(img)
        txt_feat = self.text_proj(text_emb).view(-1, 1, 4, 4)
        combined = torch.cat([img_feat, txt_feat], dim=1)
        return self.final(combined).view(-1)


def weights_init(m):
    if 'Conv' in m.__class__.__name__:
        nn.init.normal_(m.weight.data, 0.0, 0.02)
    elif 'BatchNorm' in m.__class__.__name__:
        nn.init.normal_(m.weight.data, 1.0, 0.02)
        nn.init.constant_(m.bias.data, 0)

G = Generator(Z_DIM, TEXT_DIM, GEN_FEATURES).to(DEVICE)
D = Discriminator(TEXT_DIM, DIS_FEATURES).to(DEVICE)
G.apply(weights_init); D.apply(weights_init)

print(f'Generator     params: {sum(p.numel() for p in G.parameters()):,}')
print(f'Discriminator params: {sum(p.numel() for p in D.parameters()):,}')
```

```
Generator     params: 4,625,280
Discriminator params: 2,767,648
```

10.Training Loop

```python
criterion = nn.BCEWithLogitsLoss()
optim_G   = optim.Adam(G.parameters(), lr=LR, betas=(BETA1, 0.999))
optim_D   = optim.Adam(D.parameters(), lr=LR, betas=(BETA1, 0.999))

# Fixed test prompts for consistent visualisation
fixed_z    = torch.randn(5, Z_DIM, device=DEVICE)
fixed_caps = [
    'a photo of a rose flower',
    'a beautiful sunflower with yellow petals',
    'a purple lavender flower in bloom',
    'a white daisy with soft petals',
    'a bright orange tulip flower'
]
fixed_proc = [preprocessor.preprocess(c)['final_text'] for c in fixed_caps]
fixed_emb  = embed_module.encode_and_project(fixed_proc).to(DEVICE)
```

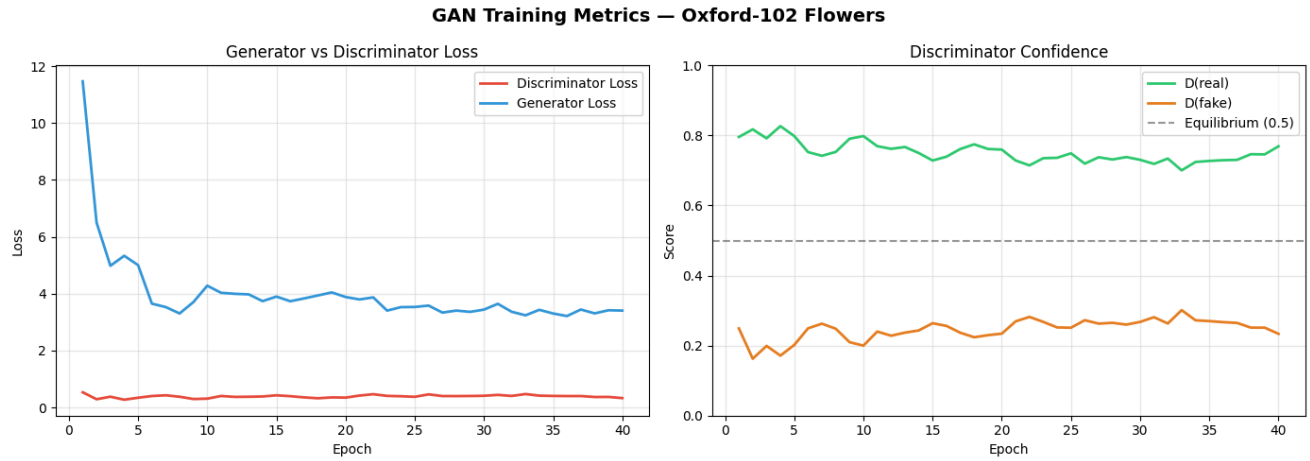
```
history      = {'d_loss': [], 'g_loss': [], 'd_real': [], 'd_fake': []}
epoch_images = []

print('Starting Training on Real Oxford-102 Images...')
print('='*60)

for epoch in range(1, EPOCHS+1):
    G.train(); D.train()
    d_losses, g_losses, d_reals, d_fakes = [], [], [], []

    for real_imgs, text_emb in dataloader:
        B         = real_imgs.size(0)
        real_imgs = real_imgs.to(DEVICE)
        text_emb  = text_emb.to(DEVICE)
        real_lbl  = torch.ones(B,  device=DEVICE)
        fake_lbl  = torch.zeros(B, device=DEVICE)

        # Train Discriminator
        D.zero_grad()
        out_real  = D(real_imgs, text_emb)
        loss_real = criterion(out_real, real_lbl)
        z         = torch.randn(B, Z_DIM, device=DEVICE)
        fake_imgs = G(z, text_emb)
        out_fake  = D(fake_imgs.detach(), text_emb)
        loss_fake = criterion(out_fake, fake_lbl)
        d_loss    = (loss_real + loss_fake) / 2
        d_loss.backward(); optim_D.step()

        # Train Generator
        G.zero_grad()
        out_gen = D(fake_imgs, text_emb)
        g_loss  = criterion(out_gen, real_lbl)
        g_loss.backward(); optim_G.step()

        d_losses.append(d_loss.item())
        g_losses.append(g_loss.item())
        d_reals.append(torch.sigmoid(out_real).mean().item())
        d_fakes.append(torch.sigmoid(out_fake).mean().item())

    history['d_loss'].append(np.mean(d_losses))
    history['g_loss'].append(np.mean(g_losses))
    history['d_real'].append(np.mean(d_reals))
    history['d_fake'].append(np.mean(d_fakes))

    if epoch % 10 == 0 or epoch == 1:
        print(f'Epoch [{epoch:3d}/{EPOCHS}]  '
              f'D_loss: {history["d_loss"][-1]:.4f}  '
              f'G_loss: {history["g_loss"][-1]:.4f}  '
              f'D(real): {history["d_real"][-1]:.3f}  '
              f'D(fake): {history["d_fake"][-1]:.3f}')

    if epoch % 10 == 0:
        G.eval()
        with torch.no_grad():
            samples = G(fixed_z, fixed_emb).cpu()
        epoch_images.append((epoch, samples))

print('\nTraining complete!')
```

```
Starting Training on Real Oxford-102 Images...
============================================================
Epoch [  1/40]  D_loss: 0.5314  G_loss: 11.4634  D(real): 0.796  D(fake): 0.249
Epoch [ 10/40]  D_loss: 0.3067  G_loss: 4.2785  D(real): 0.798  D(fake): 0.200
Epoch [ 20/40]  D_loss: 0.3438  G_loss: 3.8814  D(real): 0.760  D(fake): 0.234
Epoch [ 30/40]  D_loss: 0.4084  G_loss: 3.4383  D(real): 0.730  D(fake): 0.268
Epoch [ 40/40]  D_loss: 0.3271  G_loss: 3.4036  D(real): 0.769  D(fake): 0.234

Training complete!
```

11.Training Curves

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('GAN Training Metrics — Oxford-102 Flowers', fontsize=14, fontweight='bold')
epochs_x = range(1, EPOCHS+1)

axes[0].plot(epochs_x, history['d_loss'], label='Discriminator Loss', color='#E74C3C', lw=2)
```

```
axes[0].plot(epochs_x, history['g_loss'], label='Generator Loss',     color='#3498DB', lw=2)
axes[0].set_title('Generator vs Discriminator Loss')
axes[0].set_xlabel('Epoch'); axes[0].set_ylabel('Loss')
axes[0].legend(); axes[0].grid(True, alpha=0.3)

axes[1].plot(epochs_x, history['d_real'], label='D(real)', color='#2ECC71', lw=2)
axes[1].plot(epochs_x, history['d_fake'], label='D(fake)', color='#E67E22', lw=2)
axes[1].axhline(0.5, color='black', linestyle='--', alpha=0.4, label='Equilibrium (0.5)')
axes[1].set_title('Discriminator Confidence')
axes[1].set_xlabel('Epoch'); axes[1].set_ylabel('Score')
axes[1].legend(); axes[1].grid(True, alpha=0.3); axes[1].set_ylim(0, 1)

plt.tight_layout()
plt.savefig('training_curves.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: training_curves.png')
```




```
Saved: training_curves.png
```

12.Generated Images Progress

```
def denorm(tensor):
    return (tensor + 1) / 2

prompt_labels = ['Rose', 'Sunflower', 'Lavender', 'Daisy', 'Tulip']
n_rows = len(epoch_images)

fig, axes = plt.subplots(n_rows, 5, figsize=(14, 3*n_rows))
fig.suptitle('Generated Images — Progress Over Training (Real Oxford-102)',
             fontsize=13, fontweight='bold')

for row_i, (ep, samples) in enumerate(epoch_images):
    for col_i in range(5):
        ax  = axes[row_i][col_i] if n_rows > 1 else axes[col_i]
        img = np.clip(denorm(samples[col_i]).permute(1,2,0).numpy(), 0, 1)
        ax.imshow(img)
        if row_i == 0: ax.set_title(prompt_labels[col_i], fontsize=9, fontweight='bold')
        if col_i == 0: ax.set_ylabel(f'Epoch {ep}', fontsize=9, fontweight='bold')
        ax.axis('off')

plt.tight_layout()
plt.savefig('generated_progress.png', dpi=150, bbox_inches='tight')
plt.show()
print(' Saved: generated_progress.png')
```

```
Saved: generated_progress.png
```

13. Baseline vs Conditional GAN Comparison

```
class BaselineGenerator(nn.Module):
    """Unconditional DCGAN — no text conditioning (Baseline)"""
    def __init__(self, z_dim, features):
        super().__init__()
        self.net = nn.Sequential(
            nn.ConvTranspose2d(z_dim,      features*8, 4,1,0,bias=False), nn.BatchNorm2d(features*8), nn.ReLU(True),
            nn.ConvTranspose2d(features*8, features*4, 4,2,1,bias=False), nn.BatchNorm2d(features*4), nn.ReLU(True),
            nn.ConvTranspose2d(features*4, features*2, 4,2,1,bias=False), nn.BatchNorm2d(features*2), nn.ReLU(True),
            nn.ConvTranspose2d(features*2, features,   4,2,1,bias=False), nn.BatchNorm2d(features),   nn.ReLU(True),
```

```
            nn.ConvTranspose2d(features,   3,          4,2,1,bias=False), nn.Tanh()
        )
    def forward(self, z): return self.net(z.unsqueeze(-1).unsqueeze(-1))

class BaselineDiscriminator(nn.Module):
    def __init__(self, features):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(3,          features,   4,2,1,bias=False), nn.LeakyReLU(0.2,True),
            nn.Conv2d(features,   features*2, 4,2,1,bias=False), nn.BatchNorm2d(features*2), nn.LeakyReLU(0.2,True),
            nn.Conv2d(features*2, features*4, 4,2,1,bias=False), nn.BatchNorm2d(features*4), nn.LeakyReLU(0.2,True),
            nn.Conv2d(features*4, features*8, 4,2,1,bias=False), nn.BatchNorm2d(features*8), nn.LeakyReLU(0.2,True),
            nn.Conv2d(features*8, 1,          4,1,0,bias=False)
        )
    def forward(self, img): return self.net(img).view(-1)


BG = BaselineGenerator(Z_DIM, GEN_FEATURES).to(DEVICE)
BD = BaselineDiscriminator(DIS_FEATURES).to(DEVICE)
BG.apply(weights_init); BD.apply(weights_init)
optim_BG = optim.Adam(BG.parameters(), lr=LR, betas=(BETA1, 0.999))
optim_BD = optim.Adam(BD.parameters(), lr=LR, betas=(BETA1, 0.999))
base_hist = {'d_loss': [], 'g_loss': []}

print('Training Baseline (unconditional) GAN...')
for epoch in range(1, 16):
    bd_losses, bg_losses = [], []
    for real_imgs, _ in dataloader:
        B = real_imgs.size(0)
        real_imgs = real_imgs.to(DEVICE)
        real_lbl = torch.ones(B,  device=DEVICE)
        fake_lbl = torch.zeros(B, device=DEVICE)
        BD.zero_grad()
        z    = torch.randn(B, Z_DIM, device=DEVICE)
        fake = BG(z)
        loss = (criterion(BD(real_imgs), real_lbl) + criterion(BD(fake.detach()), fake_lbl)) / 2
        loss.backward(); optim_BD.step()
        BG.zero_grad()
        gl = criterion(BD(fake), real_lbl)
        gl.backward(); optim_BG.step()
        bd_losses.append(loss.item()); bg_losses.append(gl.item())
    base_hist['d_loss'].append(np.mean(bd_losses))
    base_hist['g_loss'].append(np.mean(bg_losses))
    if epoch % 5 == 0:
        print(f'  Epoch {epoch}/15 — D: {base_hist["d_loss"][-1]:.4f}  G: {base_hist["g_loss"][-1]:.4f}')

print(' Baseline done!')

# Comparison plot
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('Baseline vs Conditional GAN — Loss Comparison', fontsize=13, fontweight='bold')

x15 = range(1, 16)
axes[0].plot(x15, base_hist['d_loss'], label='D Loss', color='#E74C3C', lw=2, linestyle='--')
axes[0].plot(x15, base_hist['g_loss'], label='G Loss', color='#3498DB', lw=2, linestyle='--')
axes[0].set_title('Baseline (Unconditional) GAN')
axes[0].legend(); axes[0].grid(True, alpha=0.3); axes[0].set_xlabel('Epoch')

axes[1].plot(x15, history['d_loss'][:15], label='D Loss', color='#E74C3C', lw=2)
axes[1].plot(x15, history['g_loss'][:15], label='G Loss', color='#3498DB', lw=2)
axes[1].set_title('Conditional GAN (Text-Guided)')
axes[1].legend(); axes[1].grid(True, alpha=0.3); axes[1].set_xlabel('Epoch')

plt.tight_layout()
plt.savefig('model_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: model_comparison.png')
```

```
Training Baseline (unconditional) GAN...
  Epoch 5/15 — D: 0.2872  G: 4.8596
```

14.Inference — Generate from Custom Text

```
def generate_from_text(texts, G, preprocessor, embed_module, n_per_text=3):
    """Full pipeline: text → preprocess → embed → GAN → image"""
```

```
    G.eval()
    processed  = [preprocessor.preprocess(t)['final_text'] for t in texts]
    embeddings = embed_module.encode_and_project(processed).to(DEVICE)
    all_images = []
    with torch.no_grad():
        for emb in embeddings:
            emb_exp = emb.unsqueeze(0).repeat(n_per_text, 1)
            z       = torch.randn(n_per_text, Z_DIM, device=DEVICE)
            imgs    = G(z, emb_exp).cpu()
            all_images.append(imgs)
    return all_images


custom_prompts = [
    "a beautiful pink rose with soft petals",
    "a bright yellow sunflower in full bloom",
    "a delicate white daisy with long petals",
    "a vibrant purple iris flower",
]

gen_images = generate_from_text(custom_prompts, G, preprocessor, embed_module)

fig, axes = plt.subplots(len(custom_prompts), 4, figsize=(14, 4*len(custom_prompts)))
fig.suptitle('Text-to-Image Inference — Oxford-102 GAN', fontsize=14, fontweight='bold')

for row, (prompt, imgs) in enumerate(zip(custom_prompts, gen_images)):
    axes[row][0].text(0.5, 0.5, f'INPUT:\n"{prompt}"',
        ha='center', va='center', fontsize=9, wrap=True,
        transform=axes[row][0].transAxes,
        bbox=dict(boxstyle='round', facecolor='lightyellow', alpha=0.8))
    axes[row][0].axis('off')
    axes[row][0].set_title('Prompt', fontsize=9, fontweight='bold')
    for col in range(3):
        img = np.clip(denorm(imgs[col]).permute(1,2,0).numpy(), 0, 1)
        axes[row][col+1].imshow(img)
        axes[row][col+1].set_title(f'Sample {col+1}', fontsize=9)
        axes[row][col+1].axis('off')

plt.tight_layout()
plt.savefig('inference_results.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: inference_results.png')
```

15.Save Model

```
torch.save({
    'generator_state_dict'    : G.state_dict(),
    'discriminator_state_dict': D.state_dict(),
    'history': history,
    'config' : {'z_dim': Z_DIM, 'text_dim': TEXT_DIM,
                'img_size': IMG_SIZE, 'epochs': EPOCHS,
                'dataset': 'Oxford-102 Flowers (real images)'}
}, 'text_to_image_gan_oxford102.pth')

print(' Model saved: text_to_image_gan_oxford102.pth')
```

16.Summary